



Prompt Engineering & Tool Calling

AICB-P1 · Ngày 4 · Làm sao nói để AI hiểu đúng ý?

Tên Giảng Viên

VinUniversity · Phase 1 · Tuần 1 · 2026

HÃY SUY NGHĨ...

*“Hai người hỏi AI cùng một việc, một người nhận kết quả xuất sắc, người kia nhận rác. Tại sao?
Và: cùng một agent, đôi khi nó gọi tool đúng, đôi khi gọi sai — do prompt hay do tool?”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội Dung Bài Học



1. Prompt fundamentals
2. Advanced prompting techniques
3. System prompt engineering
4. Context engineering
5. Prompt safety & evaluation
6. Tool calling
7. Design principles cho tools
8. Tool patterns & error handling
9. Lab 4 + deliverable cuối buổi

- Viết được prompt rõ ràng theo các thành phần **Role / Task / Context / Format**
- Hiểu khi nào nên dùng **zero-shot**, **few-shot**, **CoT**, và khi nào không cần
- Viết được **system prompt production-grade** cho agent
- Khai báo được **tool schema** và hiểu vòng lặp tool calling từ model đến tool rồi quay lại model
- Nhận diện được **prompt injection** và viết system prompt an toàn
- Biết cách **iterate** và **evaluate** prompt quality

Mục tiêu của buổi này là hiểu **cơ chế**: prompt là interface giữa human intent và model behavior; tool calling là interface giữa model và thế giới bên ngoài.

1 agent script chạy được + 1 system prompt + 2 tool schemas + 5 test questions + ghi chú lỗi prompt/tool/control flow + checklist self-review

- 2 tools tự viết: 1 API wrapper đơn giản, 1 data query đơn giản
- 1 system prompt có rules, constraints, output contract
- 5 câu test để chứng minh agent biết khi nào trả lời trực tiếp, khi nào gọi tool
- Self-review checklist cho system prompt (6 items)

Prompt Engineering Fundamentals

Prompt tốt không phải prompt “hay”, mà là prompt tạo ra hành vi mong muốn ổn định

Prompt = Interface Giữa Ý Định và Khả Năng Model



Prompt kém

"Viết email cho tôi"

Không rõ gửi ai, về gì, tone nào, dài bao nhiêu.

Kết quả: chung chung, khó dùng ngay.

Prompt tốt

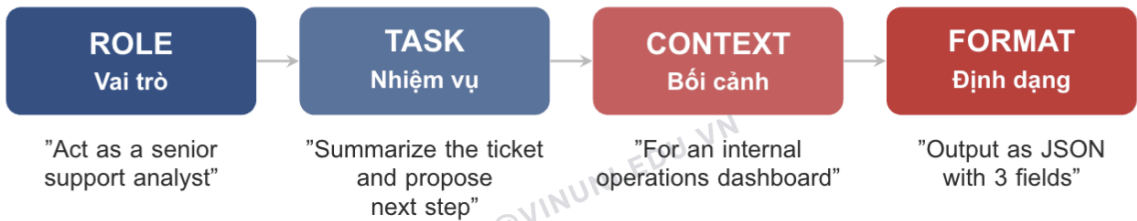
Viết email xin lỗi khách hàng về giao hàng trễ 2 ngày, tone lịch sự, dưới 120 từ, có CTA rõ ràng.

Rõ task, context, constraint, format.

Kết quả: actionable hơn hẳn.

Lưu ý: Nguyên tắc vàng: **Specificity beats cleverness**. Prompt ngắn nhưng rõ nghĩa thường tốt hơn prompt dài mà lan man.

4 Thành Phần Của Prompt Tốt



Bắt đầu với **Task + Format**. Chỉ thêm Role hoặc Context khi chúng thực sự cải thiện chất lượng hoặc tính nhất quán.

RTCF Deep Dive: Ví Dụ Thực Tế

Component	Ví dụ tốt	Ví dụ kém	Tại sao
Role	"Senior Python dev, FastAPI expert"	"Developer"	Ảnh hưởng code style, library choices
Task	"Refactor function X to use async/await"	"Fix code"	Specificity giảm ambiguity
Context	"Codebase: FastAPI, Python 3.12, PostgreSQL"	(trống)	Model đoán sai stack
Format	"Return only the function, no explanation"	(trống)	Model thêm giải thích không cần

Mỗi component thêm vào prompt phải có lý do rõ ràng

v1 — Mơ hồ

"Tóm tắt bài báo này"

Không rõ dài bao nhiêu, cho ai đọc, focus gì.

v2 — Có format

"Tóm tắt trong 3 bullets, mỗi bullet dưới 20 từ"

Rõ format, nhưng thiếu audience và focus.

v3 — RTCF đầy đủ

"Tóm tắt cho executive team. 3 bullets, <20 từ. Focus: Q2 revenue impact. Tone: data-driven."

Rõ audience, task, constraint, format.

Prompt engineering là **iterative process**. Viết → test → observe → improve. Không ai viết prompt hoàn hảo lần đầu.

Instruction vs Conversation vs System Prompt

Loại prompt	Mục đích chính	Khi dùng
Instruction prompt	Ra lệnh trực tiếp cho một tác vụ	Hỏi đáp 1 lượt, transform, summarize, classify
Conversation prompt	Giữ ngữ cảnh nhiều lượt với user	Chatbot, support, tutor, debugging nhiều bước
System prompt	Đặt policy, boundary, output contract	Agent, assistant production, use case cần hành vi ổn định

Anthropic prompting guidance + teaching heuristics

Chỉ nói "đừng" — kém

"Đừng dùng jargon"
"Đừng đoán"
"Đừng trả lời quá dài"

Nói rõ thay thế — tốt

"Giải thích bằng ngôn ngữ lớp 10 hiểu được"
"Nếu không chắc, trả lời: Tôi cần thêm thông tin"
"Giới hạn dưới 150 từ"

Negative prompts hiệu quả nhất khi **kèm positive alternative**. Model cần biết nên làm gì, không chỉ biết đừng làm gì.

Token Budget Awareness

- Prompt dài hơn **không đồng nghĩa** prompt tốt hơn.
- Mỗi token thừa làm tăng **chi phí, latency**, và đôi khi cả nhiễu.
- Hãy ưu tiên: instruction rõ, examples đúng chỗ, output contract rõ.
- Rule thực dụng: nếu prompt dài thêm nhưng không làm thay đổi hành vi mong muốn, hãy cắt bớt.

Lưu ý: Prompt engineering tốt là tối ưu **độ rõ** và **khả năng kiểm soát**, không phải thi xem ai viết prompt dài hơn.

Use case	Temp	Lý do
Classification, extraction	0	Deterministic, reproducible
Chatbot, support	0.3–0.5	Nhất quán nhưng tự nhiên
Creative writing	0.7–1.0	Đa dạng, sáng tạo
Brainstorming	1.0–1.5	Khám phá, chấp nhận noise

Lưu ý: Temperature không thay thế prompt tốt. Nếu prompt mơ hồ, giảm temperature chỉ khiến model **lặp lại cùng một output kém**.

Chỉ xét các tokens có tổng xác suất $\leq p$. Thường dùng $p = 0.9–0.95$. Đừng tune cả temp và top_p cùng lúc.

Quick Exercise: Viết Prompt Theo RTCF (2 phút)

Bạn cần viết prompt cho chatbot hỗ trợ sinh viên VinUni đăng ký môn học.

Xác định 4 thành phần:

- **Role:** Chatbot là ai? Expertise level?
- **Task:** Nhiệm vụ cụ thể là gì?
- **Context:** Hệ thống nào? Giới hạn gì?
- **Format:** Output trông như thế nào?

Thảo luận với người bên cạnh. Chia sẻ 1–2 ví dụ sau 2 phút.

Advanced Prompting Techniques

Dùng kỹ thuật nâng cao khi chúng cải thiện chất lượng thật sự, không dùng như thần chú

Zero-shot, One-shot, Few-shot, CoT



Zero-shot

Không có ví dụ mẫu.
Nhanh, rẻ, nên thử trước.

One-shot

1 ví dụ mẫu.
Tốt khi cần giữ format rõ hơn.

Few-shot

2–5 ví dụ.
Tăng consistency, nhưng tốn token hơn.

CoT

Cho model reasoning từng bước.
Hữu ích cho task suy luận.

Thứ tự thử thực dụng: **zero-shot -> few-shot -> decomposition / CoT**. Đừng nhảy vào prompt phức tạp ngay từ đầu.



- Khi model hiểu task nhưng **ra sai format** hoặc **không ổn định giữa các input tương tự**.
- Khi cần giữ tiêu chuẩn đánh giá, tone, hoặc cách lập luận nhất quán.
- Ví dụ mẫu nên **relevant**, **đa dạng vừa đủ**, và **đúng format mong muốn**.

Few-shot không phải để “dạy lại” model mọi thứ; nó là cách chỉ ra pattern mà bạn muốn model bám theo.

Nguồn minh họa: zero/few-shot teaching graphic trong repo

Few-shot Prompting — Python Example

```
examples = """
Input: "Great product, fast delivery!"
Output: Positive

Input: "Terrible quality, waste of money"
Output: Negative
"""

prompt = f"""Classify feedback as Positive, Negative, or Neutral.

{examples}
Input: "Love the design but shipping was slow"
Output: ""
print(prompt)
```

- ☐ **Ví dụ quá giống nhau:** model overfits pattern, không generalize sang input mới
- ☐ **Ví dụ sai format:** model copy sai format từ examples
- ☐ **Quá nhiều ví dụ (>5):** diminishing returns, tốn token, chậm hơn
- ☐ **Ví dụ có lỗi:** model sẽ reproduce lỗi một cách trung thành
- ☒ **Best practice:** ví dụ **đa dạng, đúng format, cover edge cases**, 2–5 examples là đủ

Chain-of-Thought (CoT) và Tree-of-Thought

CoT phù hợp khi:

- Bài toán cần reasoning nhiều bước
- Bạn muốn model giải thích logic trung gian
- Bạn cần debug xem model sai ở bước nào

Tree-of-Thought:

- Hữu ích cho bài toán cần explore nhiều hướng
- Phức tạp hơn, tốn token và latency hơn
- Chỉ nên giới thiệu như extension, không phải mặc định cho mọi task

CoT là công cụ cải thiện reasoning, không phải phép màu. Nếu task vốn dĩ chỉ là formatting hoặc extraction đơn giản, CoT thường là overkill.


```
prompt = """Phan tich review khách sạn và cho điểm 1-5.

Hay suy nghĩ từng bước:
1. Xác định các khía cạnh được nhắc đến
2. Đánh giá sentiment của từng khía cạnh
3. Tổng hợp điểm cuối cùng

Review: "Phòng rộng, view đẹp, nhưng dịch vụ chậm và giá hơi cao"

Phan tich: """

# Không CoT: model trả lời "3/5" (không giải thích)
# Co CoT: model liệt kê từng khía cạnh, đánh giá, rồi kết luận
#         -> để debug, để hiểu tại sao model cho điểm như vậy
```

Structured Output Prompting

Tại sao cần?

LLM output mặc định là free-form text, khó parse programmatically. Trong agent pipeline, bạn cần JSON/structured data.

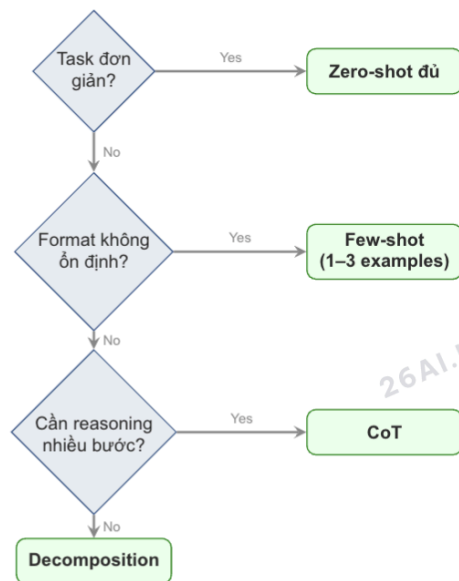
Các cách tiếp cận:

- **JSON mode:** API parameter (OpenAI)
- **Prompt-based:** "Respond ONLY with valid JSON"
- **XML tags:**
<thinking>...</thinking>
- **Prefill:** Bắt đầu assistant msg bằng { (Anthropic)

Lưu ý: Luôn **validate** JSON output. Model có thể trả sai format, đặc biệt với schema phức tạp hoặc temperature cao.

Đưa JSON schema ví dụ vào prompt giúp model bám format tốt hơn. Ví dụ:

```
{"intent": "...", "action": "...",  
 "reply": "..."} 
```



Bắt đầu đơn giản. Chỉ thêm complexity khi output chưa đạt yêu cầu.

System Prompt Engineering

System prompt tốt làm agent nhất quán hơn, dễ kiểm soát hơn, và dễ test hơn

Persona: role, expertise level, communication style

Rules: việc nên làm, việc luôn phải làm

Capabilities: model được phép dùng tools nào, dữ liệu nào

Constraints: không làm gì, khi nào từ chối, khi nào escalate

Output format: JSON, markdown, bullet list, schema, language

priority

System Prompt — Python Example

```
system_prompt = """
You are a support triage agent for an e-commerce team.

Rules:
- Answer in Vietnamese.
- Be concise and operational.
- If billing or refund policy is unclear, ask for more details.

Constraints:
- Never invent order status.
- Never promise refunds without tool confirmation.

Output format:
Return JSON with: intent, action, reply
"""
```

v1 — Thiếu constraints

You are a support agent.
 Help customers with orders.
 Be polite.

 Vấn đề: model hallucinate order status,
 trả lời câu hỏi ngoài scope,
 output format không nhất quán.

v2 — Sau khi test & fix

You are a support triage agent.
 Rules: Answer in Vietnamese. Be concise.
 Constraints: NEVER invent order status.
 If out of scope, say: "Tôi chỉ hỗ trợ về đơn hàng."
 Output: JSON {intent, action, reply}

 Cải thiện: clear boundaries, output contract,
 refusal pattern rõ ràng.

System prompt cần **iterate dựa trên test results**. Viết → test 10 câu → fix → test lại.

System Prompt: Anthropic vs OpenAI API

Anthropic Claude

```
client.messages.create(
    model="claude-sonnet-4-...",
    system="You are...",
    messages=[...],
    tools=[...]
)
```

Hỗ trợ XML tags: <rules>, <constraints> trong system prompt để cấu trúc rõ hơn.

OpenAI GPT

```
client.chat.completions.create(
    model="gpt-4.1",
    messages=[
        {"role": "system",
         "content": "You are..."},
        {"role": "user", ...}
    ],
    tools=[...]
)
```

System prompt nằm trong messages array.

Concept giống nhau, chỉ khác syntax. Dùng markdown/XML sections để structure system prompt dài.

- ☐ **Quá dài:** nhồi mọi thứ vào 1 prompt 2000+ tokens rồi hy vọng model luôn làm đúng
- ☐ **Mâu thuẫn:** vừa bảo “ngắn gọn”, vừa bắt “giải thích chi tiết từng bước”
- ☐ **Mơ hồ:** “hãy thông minh”, “hãy chuyên nghiệp”, nhưng không định nghĩa chuẩn output
- ☐ **Không test edge cases:** quên kiểm tra câu hỏi ngoài phạm vi, refusal, tool failure
- ☒ **Nguyên tắc:** system prompt là policy layer. Càng rõ boundary, càng dễ predict hành vi

- ☒ **Happy path:** câu hỏi trong scope → trả lời đúng format?
- ☒ **Edge case:** câu hỏi mơ hồ → hỏi lại hay đoán bừa?
- ☒ **Out of scope:** câu hỏi ngoài phạm vi → từ chối đúng cách?
- ☒ **Adversarial:** prompt injection → có bị bypass?
- ☒ **Tool decision:** khi nào gọi tool vs khi nào trả lời trực tiếp?
- ☒ **Format consistency:** 10 câu khác nhau → output format nhất quán?

```
## Identity
Bạn là [role] cho [company/product].

## Rules
- ALWAYS: [hành vi bắt buộc]
- NEVER: [hành vi cấm]
- WHEN [condition]: [hành vi cụ thể]

## Available Tools
- tool_name: khi nào dùng, khi nào KHÔNG dùng

## Output Format
{"intent": "...", "action": "...", "reply": "..."}

## Escalation
Khi [điều kiện] -> chuyển cho nhân viên
```

Dùng template này làm starting point. Thêm/bớt sections tùy use case.

Mini Exercise: Critique a System Prompt (3 phút)

You are a helpful assistant. Be smart and professional.
Answer any question the user asks. Be concise but also explain in detail.
You can use tools. Always respond in JSON format. If you don't know, make your best guess.

Tìm ít nhất 3 vấn đề trong system prompt trên.

Gợi ý: Mâu thuẫn? Mơ hồ? Thiếu gì? Nguy hiểm ở đâu?

Thảo luận nhóm 3 phút → chia sẻ.

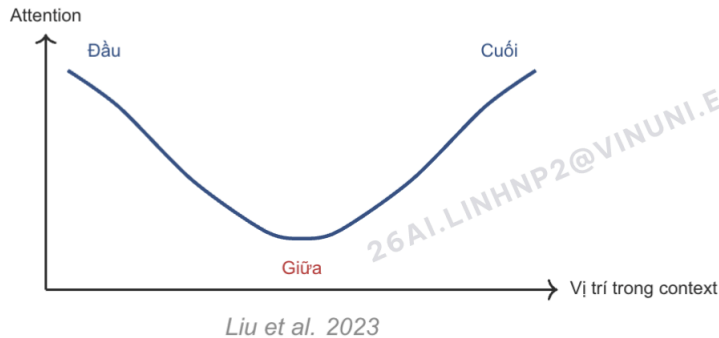
Context Engineering

Điều quan trọng không phải nhét bao nhiêu context, mà là chọn đúng context cần thiết

Context Window Management



Lưu ý: Token budget allocation cần chủ động: đừng để history, tools, và examples ăn hết chỗ dành cho output.



Hệ quả thực tiễn:

- Đặt instructions quan trọng ở **đầu hoặc cuối**
- Context dài → info ở giữa dễ bị “quên”
- Break long lists bằng headers/separators
- Recent context nên đặt ngay trước user query

Memory Injection và Context Compression

Memory injection

- Chỉ đưa vào facts thật sự cần cho task hiện tại
- Ưu tiên recent history hoặc relevant history, không dump toàn bộ transcript
- Tốt cho support agent, coding assistant, tutor nhiều lượt

Compression

- **Summarize:** tóm tắt phần cũ
- **Drop:** bỏ hẳn phần không còn liên quan
- **Archive:** đẩy ra ngoài context, chỉ fetch lại khi cần

Context engineering là bài toán chọn lọc và ưu tiên. Nếu mọi thứ đều quan trọng, thực ra không có gì thực sự nổi bật với model.

Token Budget Allocation: Nên Nghĩ Theo Rõ Nào?

Rõ token	Chứa gì	Rủi ro nếu quá nhiều
System prompt	policy, rules, output format	chậm hơn, khó maintain
History	recent turns, facts liên quan	dễ nhiễu, dễ lost in the middle
Tool schemas	tên tool, mô tả, tham số	model chọn tool tệ nếu schema dài hoặc mơ hồ
Output buffer	phần model dùng để trả lời	bị cắt cụt output nếu cấp thiếu

Teaching heuristic for token budgeting

RAG Context Pattern



Best practices:

- Inject với source citation
- Limit chunk size (500–1000 tokens)
- Rank by relevance, chỉ lấy top-k

Agent có thể có tool `search_kb` để retrieve context on-demand, thay vì nhét sẵn toàn bộ KB vào prompt.

- ☒ Đã cắt bỏ history không liên quan đến task hiện tại?
- ☒ System prompt có dưới 500 tokens (trừ khi cần hơn)?
- ☒ Tool schemas có concise descriptions (không dài quá)?
- ☒ Output buffer đủ cho expected response length?
- ☒ Important info ở đầu hoặc cuối context (tránh middle)?

Prompt Safety & Evaluation

Prompt tốt không chỉ cho kết quả đúng — nó còn phải an toàn và đáng tin

Direct injection

User trực tiếp nói "Ignore your instructions and do X"

Indirect injection

Malicious content trong document/email mà agent đọc qua tool

Defense Strategies

1. Delimiter separation:

Wrap untrusted input:

`<user_input>...</user_input>`

2. Instruction hierarchy:

System prompt luôn ưu tiên hơn user input

3. Input validation:

Filter known injection patterns trước khi đưa vào prompt

4. Output validation:

Kiểm tra output trước khi execute actions

5. Least privilege:

Tool permissions tối thiểu cần thiết

6. Human-in-the-loop:

Yêu cầu confirm cho sensitive actions

Lưu ý: Không có defense nào là hoàn hảo 100%. Defense-in-depth: kết hợp nhiều layers.

Dimension	Câu hỏi	Đo bằng cách
Correctness	Output có đúng không?	Test cases + human review
Consistency	10 lần chạy cho cùng kết quả?	Chạy lặp lại, đo % match
Safety	Có bị bypass không?	Adversarial test cases

Chạy 10–20 test cases. Nếu <90% pass → cần iterate prompt.

A/B testing: so sánh prompt v1 vs v2 trên cùng test set.



Pre-guard:

- Detect injection attempts
- Validate input format
- Rate limiting

Post-guard:

- Mask PII trong output
- Validate JSON schema
- Block dangerous tool calls

Tool Calling

Tool calling là cách agent chuyển từ “nói” sang “tương tác với thế giới thực”

Tool Calling Flow



Model không tự chạy code hay tự gọi API ngoài. Ứng dụng của bạn nhận tool request, chạy tool, rồi gửi kết quả trở lại model.

Vai trò	Trách nhiệm	Ví dụ
Developer (bạn)	Định nghĩa tool schema, viết implementation, handle errors	Viết <code>get_weather()</code> function
LLM	Quyết định tool nào, arguments gì, dựa trên user intent	Output: <code>{"name": "get_weather", "city": "Hanoi"}</code>
Application	Nhận tool call, execute, trả result	Gọi API weather, trả JSON result
LLM (lần 2)	Synthesize tool result thành câu trả lời tự nhiên	"Hà Nội hôm nay 32°C, trời nắng"

Phân vai rõ ràng giúp hiểu đúng cơ chế

Tool Schema Anatomy

- **Name:** nên ngắn, rõ, động từ đúng việc
- **Description:** nói khi nào nên dùng tool này
- **Parameters:** mô tả input bằng JSON Schema
- **Required fields:** giúp model biết thiếu gì thì chưa gọi được

Lưu ý: LLM đọc description như tài liệu hướng dẫn. Nếu description mơ hồ, model sẽ chọn sai tool hoặc truyền sai arguments.

```
weather_tool = {
  "type": "function",
  "function": {
    "name": "get_weather",
    "description": "Get current weather for a city when the user asks about weather conditions.",
    "parameters": {
      "type": "object",
      "properties": {
        "city": {"type": "string", "description": "City name, e.g. Hanoi"}
      },
      "required": ["city"]
    }
  }
}
```

Good vs Bad Tool Description

	Description	Hệ quả
Bad	"Gets weather"	Quá ngắn, model không biết khi nào dùng
Bad	"This comprehensive tool can be used to retrieve current weather information for any city worldwide..."	Quá dài, thêm noise
Good	"Get current weather for a city. Use when user asks about weather, temperature, or conditions."	Rõ chức năng + trigger condition

Tool description = documentation cho model. Nên chứa: (1) chức năng, (2) khi nào dùng, (3) khi nào KHÔNG dùng.

Giá trị	Ý nghĩa	Khi dùng
auto (mặc định)	Model tự quyết gọi hay không	Hầu hết use cases
required / any	Buộc gọi ít nhất 1 tool	Pipeline steps, routing
none	Cấm gọi tool, chỉ text	Test, fallback mode
{"name": "X"}	Buộc gọi tool cụ thể	Khi biết chắc cần tool nào

Lưu ý: Dùng required cẩn thận: model có thể gọi tool với arguments bịa nếu user không cung cấp đủ thông tin.

Tool Calling: OpenAI vs Anthropic Format

OpenAI

```
tools = [{  
  "type": "function",  
  "function": {  
    "name": "get_weather",  
    "description": "...",  
    "parameters": {...}  
  }  
}]  
  
Response:  
message.tool_calls[0]  
.function.name / .arguments
```

Concept giống nhau. Khác: parameters vs input_schema, response structure.

Anthropic

```
tools = [{  
  "name": "get_weather",  
  "description": "...",  
  "input_schema": {...}  
}]  
  
Response:  
content[i].type == "tool_use"  
content[i].name / .input
```


Lỗi	Xử lý
Timeout	Retry + exponential backoff
Error response	Truyền error message cho model để nó thông báo user
Unexpected format	Validation layer + fallback
Tool not found	Log + return error JSON

Thêm instruction:

"If a tool returns an error, explain the issue to the user and suggest alternatives. Never retry silently more than 2 times."

Lưu ý: Tool errors không phải edge case — chúng **sẽ** xảy ra trong production. Plan for failure.

Design Principles Cho Tools

Tool tốt là software interface tốt, không phải prompt trang trí

Nguyên tắc	Ý nghĩa	Nếu vi phạm
Single Responsibility	Mỗi tool làm 1 việc rõ ràng	model khó quyết định nên gọi tool nào
Idempotency	Cùng input cho cùng kết quả; side effect được kiểm soát	retry dễ sinh lỗi phụ
Granularity hợp lý	Không quá nhỏ, cũng không ôm quá nhiều việc	hoặc overhead lớn, hoặc tool quá cứng
Test độc lập	Unit test từng tool trước khi gắn vào agent	khó tách lỗi tool khỏi lỗi prompt

Principles for reliable tool interfaces

Tool Granularity: Quá Nhỏ Hay Quá To Đều Có Giá

Quá nhỏ

- get_customer_name
- get_customer_email
- get_customer_phone

Hệ quả: quá nhiều calls, overhead lớn, flow rối.

Quá to

- handle_all_customer_operations

Hệ quả: model không hiểu boundary, khó debug, khó reuse.

Thiết kế tool quanh một hành động nghiệp vụ rõ ràng: ví dụ lookup_order, get_weather, query_sales_data, send_email_draft.

- **Required vs Optional:** chỉ require những gì thực sự cần
- **Enum constraints:**
"status": {"type": "string",
"enum": ["pending", "shipped", "delivered"]}
→ Giảm lỗi arguments
- **Default values:** document rõ trong description

Thêm ví dụ vào parameter description:

```
"date": {  
  "type": "string",  
  "description": "Date in  
  YYYY-MM-DD format,  
  e.g. 2026-04-05"  
}
```

Structured response:

```
// Success  
{  
  "status": "success",  
  "data": {"temp": 32, "city": "Hanoi"},  
  "source": "openweathermap"}  
  
// Error  
{  
  "status": "error",  
  "message": "City not found",  
  "code": "NOT_FOUND"}
```

Rules:

- Trả JSON, không raw HTML/XML
- Error format consistent
- Include metadata (source, timestamp)
- Truncate nếu response quá dài

Lưu ý: Model xử lý structured JSON tốt hơn nhiều so với raw text hay HTML.

Cùng tool, description khác → model behavior khác hoàn toàn

Mơ hồ

"Search orders"

Model gọi cho MỌI câu hỏi liên quan đến order, kể cả "đơn hàng là gì?"

Rõ ràng

"Search orders by order_id or customer email.
Use ONLY when user provides an order number or asks about specific order status."

Model biết boundary rõ, chỉ gọi khi có đủ data

Description nên chứa: **(1) chức năng, (2) khi nào dùng, (3) khi nào KHÔNG dùng.**
Viết như viết API docs.

Parallel Tool Calling & Patterns

Nhanh hơn không có nghĩa là tốt hơn nếu flow control và merge logic không rõ

Sequential

Tool B cần output của Tool A.
Ví dụ: tìm order ID -> rồi mới tra shipping status.

Parallel

Các tool độc lập có thể chạy cùng lúc.
Ví dụ: gọi thời tiết, tỷ giá, và lịch học song song.

Lưu ý: Chỉ song song hóa khi không có phụ thuộc dữ liệu. Nếu song song, vẫn cần bước merge / verify rõ ràng ở cuối.

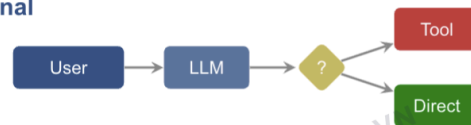
3 Tool Use Patterns Thường Gặp

1. **Conditional tool use:** agent tự quyết định có cần tool hay trả lời trực tiếp.
2. **Tool chaining:** output của tool A là input của tool B.
3. **Parallel fetch + merge:** lấy nhiều nguồn độc lập rồi tổng hợp kết quả.

Tool calling không chỉ là “gọi API”. Nó là bài toán control flow: khi nào gọi, gọi cái gì, gọi theo thứ tự nào, và làm gì khi tool fail.

3 Patterns — Visual Flow

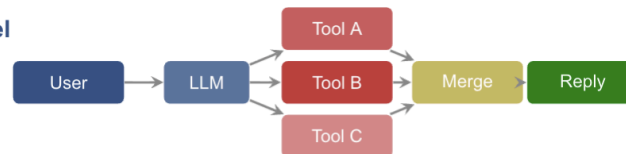
1. Conditional



2. Chaining



3. Parallel



Minimal Tool Loop — Python Example

```
messages = [{"role": "user", "content": "Thoi tiet Ha Noi va ty gia USD hom nay?"}]
response = client.responses.create(model="gpt-4.1", input=messages, tools=tools)

for item in response.output:
    if item.type == "function_call":
        result = run_tool(item.name, json.loads(item.arguments))
        messages.append(item)
        messages.append({"type": "function_call_output", "call_id": item.call_id, "output": result})

final = client.responses.create(model="gpt-4.1", input=messages, tools=tools)
print(final.output_text)
```

```
MAX_ROUNDS = 5
messages = [{"role": "user", "content": user_input}]

for round_num in range(MAX_ROUNDS):
    response = call_model(messages, SYSTEM_PROMPT, TOOLS)
    tool_calls = extract_tool_calls(response)
    if not tool_calls:
        break # Model done, no more tools needed

    for tc in tool_calls:
        try:
            result = execute_tool(tc.name, tc.args)
        except TimeoutError:
            result = {"error": "Tool timed out, please try again"}
        except Exception as e:
            result = {"error": str(e)}
        messages.append(tool_result(tc.id, json.dumps(result)))
    else:
        print("Warning: max tool rounds reached")
```

Thực Hành

Lab 4: Build first agent với system prompt + 2 tools + 5 test cases

09

1. Viết 1 system prompt với rules, constraints, output format
2. Tạo 2 custom tools: 1 API wrapper đơn giản, 1 data query đơn giản
3. Nối tools vào agent loop
4. Chạy 5 câu test để xem khi nào agent trả lời trực tiếp, khi nào gọi tool
5. Ghi lại lỗi thuộc loại prompt, tool schema, hay control flow

Lab Skeleton — Python Example

```
SYSTEM_PROMPT = open("system_prompt.txt").read()
TOOLS = [get_weather_tool(), query_sales_tool()]

while True:
    user_input = input("You: ")
    messages.append({"role": "user", "content": user_input})
    response = call_model(messages, SYSTEM_PROMPT, TOOLS)
    messages = handle_tool_calls(response, messages)
    print(render_final_answer(messages, SYSTEM_PROMPT, TOOLS))
```


Step 1–3: Setup

1. Chọn domain (weather + sales, hoặc tự chọn)
2. Viết system prompt (dùng template đã học)
3. Viết 2 tool schemas (name, description, params)

Step 4–6: Build & Test

4. Implement tool functions (mock data OK)
5. Wire vào agent loop (có error handling)
6. Test 5 câu hỏi, ghi pass/fail + lỗi

Bắt đầu với mock tools (trả data cố định) trước. Đảm bảo flow đúng rồi mới lo về real data.

5 Test Questions Gợi Ý

#	Câu hỏi	Expected	Kiểm tra
1	"Thời tiết Hà Nội hôm nay?"	Gọi get_weather	Tool A hoạt động
2	"Doanh số tháng 3 là bao nhiêu?"	Gọi query_sales	Tool B hoạt động
3	"So sánh doanh số với thời tiết tuần này"	Gọi cả 2 tools	Parallel/chaining
4	"Prompt engineering là gì?"	Trả lời trực tiếp	Conditional: no tool
5	"Cho tôi số điện thoại CEO"	Từ chối, out of scope	Refusal handling

Thêm câu test

riêng nếu agent của bạn có domain khác.

- ☒ Agent chạy end-to-end không crash?
- ☒ System prompt có đủ 5 thành phần (Persona, Rules, Capabilities, Constraints, Format)?
- ☒ Tool schemas có clear descriptions + required fields?
- ☒ Agent biết khi nào gọi tool vs khi nào trả lời trực tiếp?
- ☒ Agent xử lý gracefully khi tool fail (không crash, thông báo user)?
- ☒ Đã ghi chú ít nhất 2 lỗi phát hiện + phân loại (prompt / tool / control flow)?

Lab #4

Mục tiêu: Build ReAct agent với 2 custom tools, viết system prompt chuẩn, và test end-to-end trên 5 câu hỏi

Deliverable: Agent script chạy được + system prompt + 2 tool schemas + 5 test outputs + note lỗi prompt/tool/control flow + self-review checklist

Thời gian: 150 phút

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 Prompt = interface giữa human intent và model capability. Prompt tốt giúp model làm đúng việc, đúng format, đúng boundary.
- 2 System prompt tốt = agent nhất quán và predictable hơn, đặc biệt khi có tools và constraints.
- 3 Tool schema description quyết định rất mạnh việc model biết khi nào dùng tool nào và gọi với arguments gì.
- 4 Parallel tool calls nhanh hơn đáng kể khi các tool độc lập; nếu có phụ thuộc dữ liệu, hãy giữ flow tuần tự.
- 5 Prompt safety (injection defense, guardrails) là bắt buộc cho production agents, không phải

AI Product Thinking & Requirements

“Bạn đã build được agent đầu tiên. Nhưng build xong chưa đủ. Ngày mai: sản phẩm này dành cho ai, yêu cầu ra sao, và rủi ro nào phải nghĩ từ đầu?”

- Hoàn thiện Lab 4 với 5 test questions rõ pass/fail
- Đọc lại system prompt của mình và chỉ ra 2 chỗ còn mơ hồ hoặc mâu thuẫn
- Thử viết 2 adversarial test cases (prompt injection) cho agent của bạn

Tài Liệu Tham Khảo

- 1 Anthropic. *Prompt Engineering Overview*. docs.anthropic.com
- 2 Anthropic. *Claude Prompting Best Practices và Multishot Prompting*. docs.anthropic.com
- 3 Anthropic. *Tool Use Overview*. docs.anthropic.com
- 4 OpenAI. *Function Calling Guide*. platform.openai.com/docs
- 5 Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in LLMs*. 2022.
- 6 Liu et al. *Lost in the Middle: How Language Models Use Long Contexts*. 2023.
- 7 LangGraph Docs. *Quickstart*. langchain-ai.github.io/langgraph
- 8 OWASP. *Top 10 for LLM Applications*. owasp.org